

NYU Fall Semester 2023

Lecture 12: Concurrency

Professor G. Sandoval



Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, and Junfeng Yang from Columbia

Outline

- From Last Class
- Implementing Locks
- Reader-writer locks

Recall: Banking example

```
int balance = 0;
int main()
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, deposit, (void*)1);
    pthread_create(&t2, NULL, withdraw, (void*)2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("all done: balance = %d\n", balance);
    return 0;
}
```

```
void* deposit(void *arg)
{
    int i;
    for(i=0; i<1e7; ++i)
        ++ balance;
}
```

```
void* withdraw(void *arg)
{
    int i;
    for(i=0; i<1e7; ++i)
        -- balance;
}
```

Lecture 12: What will the final balance be?

Always 0

0%

Sometimes 0

0%

Random

0%

Compiler Bug

0%

Results of the banking example

```
$ gcc -Wall -lpthread -o bank bank.c
```

```
$ bank
```

```
all done: balance = 0
```

```
$ bank
```

```
all done: balance = 140020
```

```
$ bank
```

```
all done: balance = -94304
```

```
$ bank
```

```
all done: balance = -191009
```

Why?

A closer look at the banking example

```
$ objdump -d bank
```

```
...
```

```
08048464 <deposit>:
```

```
...
```

```
//++ balance
```

```
8048473: a1 80 97 04 08
```

```
mov 0x8049780,%eax
```

```
8048478: 83 c0 01
```

```
add $0x1,%eax
```

```
804847b: a3 80 97 04 08
```

```
mov %eax,0x8049780
```

```
...
```

```
0804849b <withdraw>:
```

```
...
```

```
//++ balance
```

```
80484aa: a1 80 97 04 08
```

```
mov 0x8049780,%eax
```

```
80484af: 83 e8 01
```

```
sub $0x1,%eax
```

```
80484b2: a3 80 97 04 08
```

```
mov %eax,0x8049780
```

One possible schedule

Time

↓

CPU 0		CPU 1
	balance: 0	
mov	0x8049780,%eax	
	eax0: 0	
add	\$0x1,%eax	
	eax0: 1	
mov	%eax,0x8049780	
	balance: 1	
		mov 0x8049780,%eax
	eax1: 1	
		sub \$0x1,%eax
	eax1: 0	
		mov %eax,0x8049780
	balance: 0	

One deposit and one withdraw, balance unchanged. **Correct**

Another possible schedule

Time ↓

CPU 0		CPU 1
	balance: 0	
mov 0x8049780,%eax		
	eax0: 0	
add \$0x1,%eax		
	eax0: 1	
		mov 0x8049780,%eax
	eax1: 0	
mov %eax,0x8049780		
	balance: 1	
		sub \$0x1,%eax
	eax1: -1	
		mov %eax,0x8049780
	balance: -1	

One deposit and one withdraw, balance become less. **Wrong!**

Race Conditions

- If *two processes* or *threads* need to update some data at the same time, we may have a *race condition*
- These problems are difficult to debug – problem only occurs when the timing is just right and is hard to reproduce
- Note that we don't actually need multiple processors here for a mistake to occur!
- Just *preemption* at the wrong time

Issues in Concurrency

- We want to run things *concurrently* for performance reasons
 - Particularly if we have multiple processors (even phones now typically have 2+ CPU cores)
 - Some of the Intel i7s have 6 cores. (i7-990x)
- But when multiple concurrent tasks (processes or threads) need to operate on some shared resources, things can get messy

Concurrency

- Theory



Concurrency

- Practice



Race Conditions

- If *two processes* or *threads* need to update some data at the same time, we may have a *race condition*
- These problems are difficult to debug – problem only occurs when the timing is just right and is hard to reproduce
- Note that we don't actually need multiple processors here for a mistake to occur!
- Just *preemption* at the wrong time

Mutual Exclusion

- The key trouble we ran into was that the *read-update-write sequence* on a shared resource could be interleaved between two processes
(read1-read2-update2-write2-update1-write1)
- The way to solve this is through *mutual exclusion* –making sure that *only one process* has access to the shared resource at a time

Critical Regions/Sections

- We can formulate the problem by classifying what programs do into two parts
 1. The majority of the time they do things **that don't require synchronization**; the things they do only affect non-shared resources
 2. Some of the time, they need to access shared memory or files; we call this a **critical region** or **critical section** of the program
- If we can arrange it so that two programs are **never in a critical section at the same time**, we can avoid race conditions

Avoid race conditions

- **Critical section**: a segment of code that accesses a shared variable (or resource)
- No more than one thread in critical section at a time.

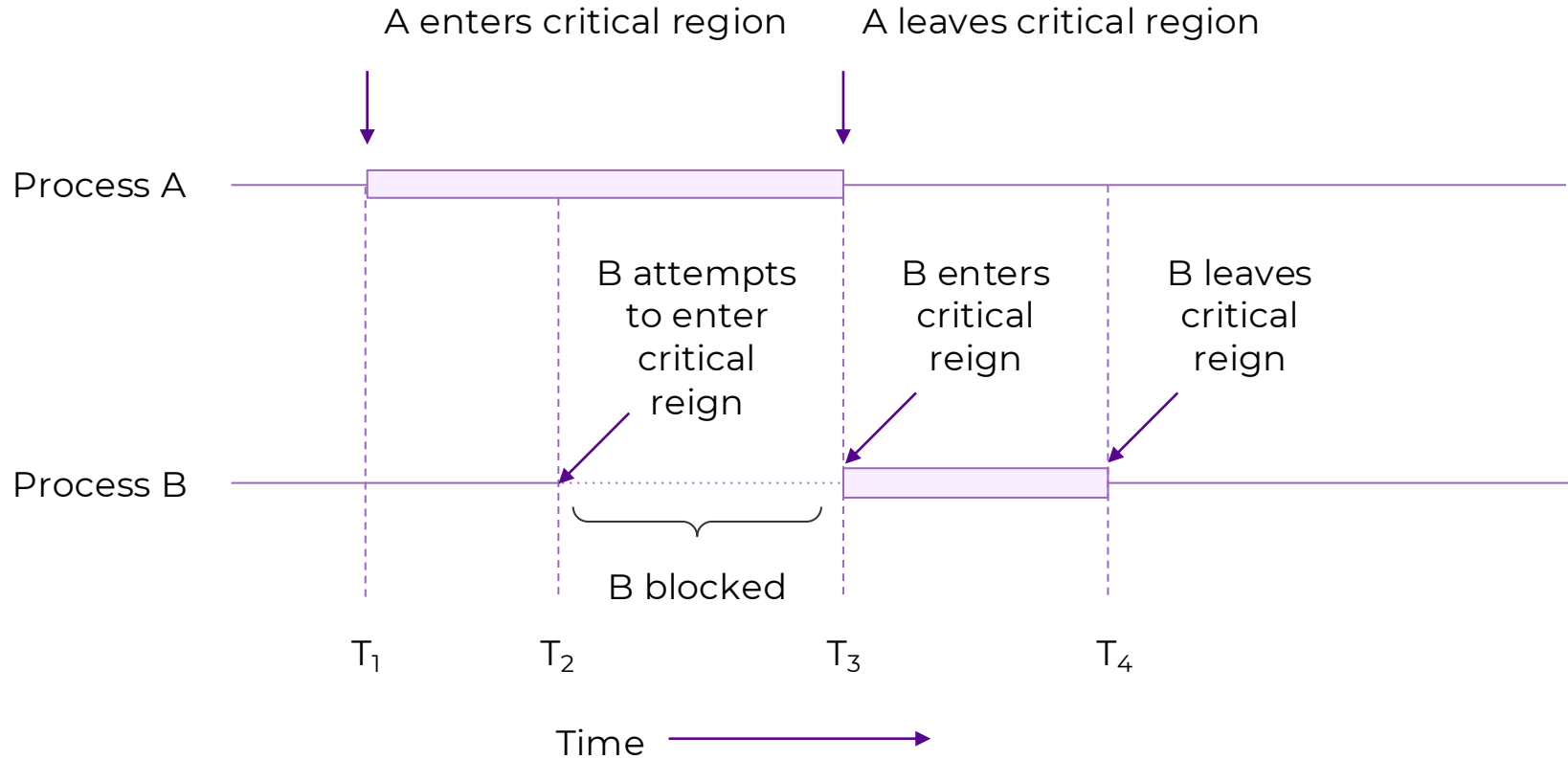
```
// ++ balance
mov     0x8049780,%eax
add     $0x1,%eax
mov     %eax,0x8049780
...
// -- balance
mov     0x8049780,%eax
sub     $0x1,%eax
mov     %eax,0x8049780
...
```

Critical Section Requirements

- **Safety (aka mutual exclusion)**: no more than one thread in critical section at a time.
- **Liveness (aka progress)**:
 - If multiple threads simultaneously request to enter critical section, must allow one to proceed
 - Must not depend on threads outside critical section
- **Bounded waiting (aka starvation-free)**
 - Must eventually allow waiting thread to proceed
- **Makes no assumptions about the speed and number of CPU**
 - However, assumes each thread makes progress

CS desirable properties

- **Efficient:** don't consume too much resource while waiting
 - Don't busy wait (spin wait). Better to relinquish CPU
 - and let other thread run
- **Fair:** don't make one thread wait longer than others. Hard to do efficiently
- **Simple:** should be easy to use



Outline

- From Last Class
- **Implementing Locks**
- Reader-writer locks

Disabling Interrupts

- If we have only one processor, the only time things can go wrong is if we get preempted in the middle of our critical section
- Preemption depends on an **interrupt** (e.g., the timer interrupt) occurring
- **Simple idea**: disable interrupts in critical sections
 - Why is this a bad idea?

Implementing locks: version 1

- Can cheat on uniprocessor: implement locks by disabling and enabling interrupts

```
lock ()                                unlock ()
{                                      {
    disable_interrupt ();              enable_interrupt ();
}
```

- Good: simple!
- Bad:
 - Both operations are privileged, can't let user program use
 - Doesn't work on multiprocessors

Lock Variable

```
void acquire(struct spinlock *lk) {  
    for(;;) {  
        if(!lk->locked) {  
            lk->locked = 1;  
            break;  
        }  
    }  
}
```

- To enter a critical region, wait until the lock variable is 0, then set it to 1
- To leave the critical region, just set the lock back to 0
- Does this work?

Class 12: Does this work?

YES

0%

NO

0%

NO, it doesn't work

Same reason as banking example.

Implementing locks: version 2

- **Peterson's algorithm**: software-based lock implementation
- **Good**: doesn't require much from hardware
 - Only assumptions:
 - Loads and stores are **atomic**
 - They execute **in order**
 - **Does not require** special hardware instructions
- **Bad**:
 - Hard to get right

Peterson's Algorithm

- Each process indicates its *interest* by setting its entry in the "interested" array
- Then, it sets the global *turn* variable to its own process number
- Finally, loop until *turn* indicates that it's our turn *and* we see that the other process is no longer interested
- There is still a race – but *regardless of the winner*, only *one process* will get to enter its critical region

Peterson's Algorithm

```
#define FALSE 0
#define TRUE 1
#define N 2
int turn;
int interested[N];
void enter_region(int process);
{
    int other;
    other = 1 - process;
    interested[process] = TRUE;
    turn = process;
    while (turn == process && interested[other] == TRUE) {

    }
}
void leave_region(int process) {
    interested[process] = FALSE;
}

/* number of processes */
/* whose turn is it? */
/* all values initially 0 (FALSE) */
/* process is 0 or 1 */

/* number of the other process */
/* the opposite of process */
/* show that you are interested */
/* set flag */
/* Do nothing

/* process: who is leaving */
/* indicate departure from
    critical region */
```

Peterson's Algorithm

- Each process indicates its *interest* by setting its entry in the "interested" array.
- Then, it sets the global *turn* variable to its own process number
- Finally, loop until *turn* indicates that it's our turn, and we see that the other process is no longer interested
- There is still a race – but regardless of the winner, only one process will get to enter its critical region

Hardware Support

- We can have a simpler solution if the hardware helps us out a bit with an *atomic instruction*
- For example, "test and set lock": **TSL RX, LOCK**
 - **Atomically** reads the memory at address LOCK into RX and then stores a nonzero value back into LOCK
 - No other processor is allowed to access the memory at address LOCK until TSL is done

Using TSL for Locks

enter_region:

```
TSL REGISTER, LOCK    | copy lock register and set lock to 1
CMP REGISTER, #0       | was lock zero?
JNE enter_region      | if it was not zero, lock was set, so loop
RET                   | return to caller; critical region entered
```

leave_region:

```
MOVE LOCK, #0         | store a 0 in lock
RET                   | return to caller
```

x86 Atomic Locking

- A similar instruction exists on x86:
xchg REG, MEM
- Atomically exchanges the contents of a register and a memory location
- You can see that this is equivalent to TSL if the register is set to 1

xv6 Lock using XCHG

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    if(holding(lk))
        panic("acquire");
    // The xchg is atomic.
    // It also serializes, so that reads after acquire are not
    // reordered before it.
    while(xchg(&lk->locked, 1) != 0)
        ;
    // Record info about lock acquisition for debugging.
    lk->cpu = cpu;
    getcallerpcs(&lk, lk->pcs);
}
```

Busy Waiting

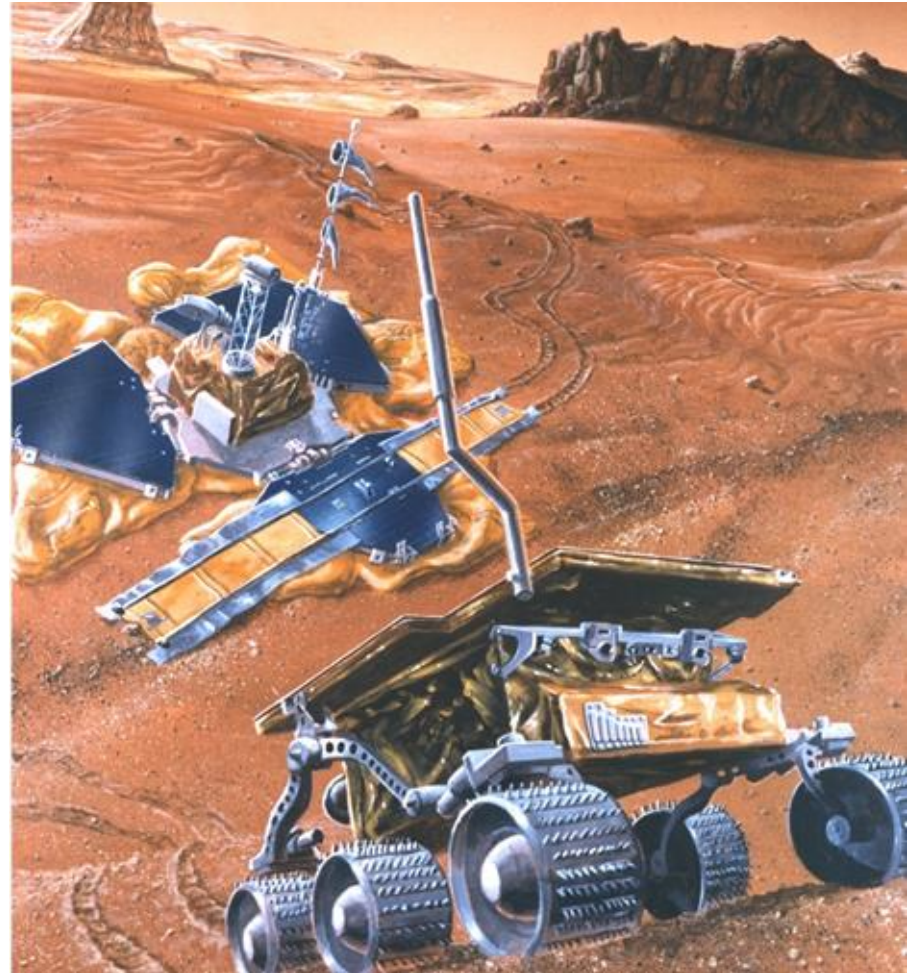
- Whenever the a process is waiting to enter a critical section under these schemes, it sits in an infinite loop
- This wastes CPU time
- It can also interact badly with scheduling

Priority Inversion

- Suppose we're using priority scheduling, and we have a high-priority process **H** and a low priority process **L**
 - So whenever **H** is runnable, it will always be chosen over **L**
- Now, **L** enters a critical region, but is then preempted to run **H**
- **H** wants to enter the critical region, but the lock is already held by **L**, so it enters a busy wait loop
- But now **H** is *always* runnable, and will *always* be chosen over **L**, so **L** can never leave its critical region and the system is stuck

The Martian Inversion

- The Mars Pathfinder mission used the VxWorks realtime operating system with priority scheduling
- Meteorological data gathering was low-priority task, communication task was medium-priority
- Both shared a data bus, controlled by a lock
- This caused a classic priority inversion, hanging the lander until the watchdog timer reset it



The Martian Inversion

- This was debugged from 140 million miles away by examining system log data
- Fixed by uploading a snippet of C code that turned on priority inheritance for the lock
- **Priority inheritance** says that a process holding a lock is elevated to the highest priority of anything waiting for the lock



Mutexes

- A **mutex** is a way to have mutual exclusion without busy waiting
- Implementation is very similar to the busy-wait version of critical regions, but instead of looping, we yield the CPU
- **Mutex** = **spinlock** + **sleep**
- When to prefer Mutex over spinlock?

Mutex Code

mutex_lock:

```
TSL REGISTER,MUTEX    | copy mutex to register and set mutex to 1
CMP REGISTER,#0        | was mutex zero?
JZE ok                | if it was zero, mutex was unlocked, so return
CALL thread_yield      | mutex is busy; schedule another thread
JMP mutex_lock         | try again
```

ok:

```
RET                    | return to caller; critical region entered
```

mutex_unlock:

```
MOVE MUTEX,#0          | store a 0 in mutex
RET                     | return to caller
```

Big Picture

- **Race conditions** → incorrect results
- **Locks** → fix correctness
- **Spinlocks** → waste CPU
- **Mutex** → better, but slower
- **Systems = tradeoffs**